
# Django Authentication

**Estimated time needed:** 30 minutes

## Learning Objectives

- Understand the Django authentication system
- Create views and templates for user log in and log out
- Create views and templates for user registration

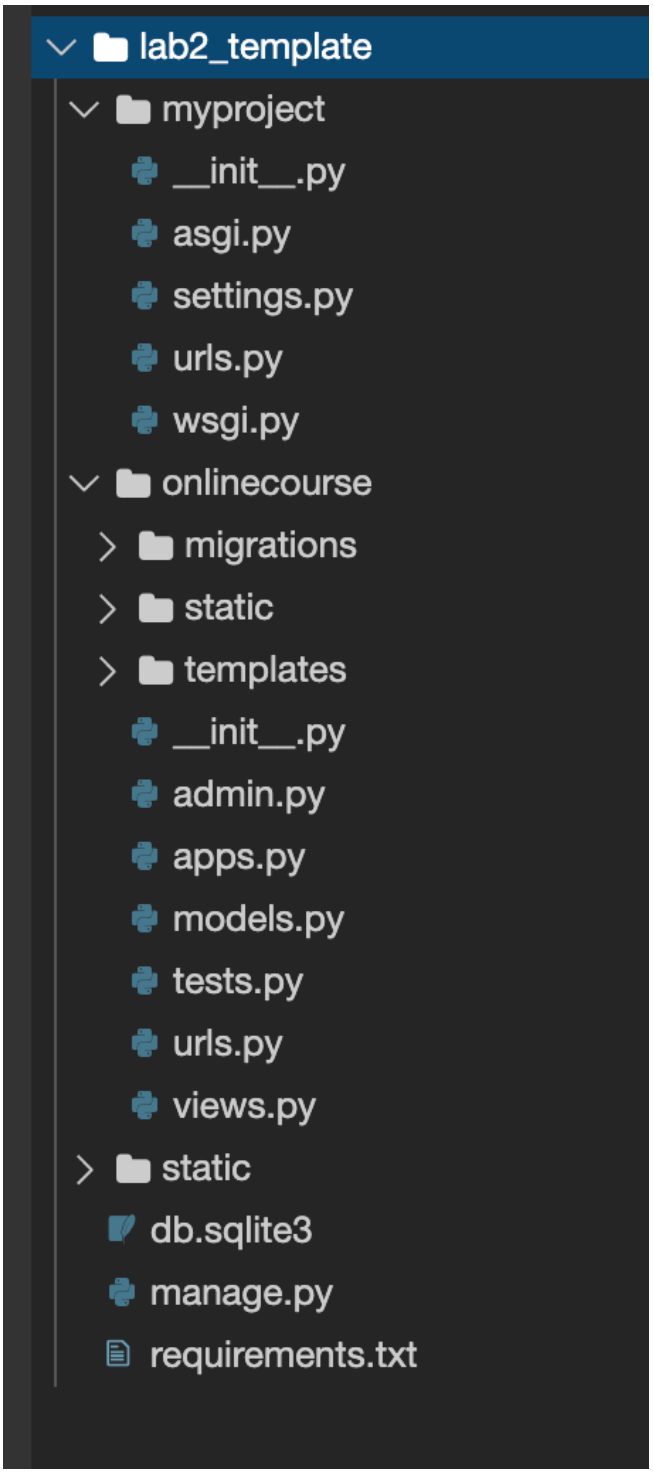
# Import an `onlinecourse` App Template and Database

If the terminal was not open, go to `Terminal > New Terminal` and make sure your current Theia directory is `/home/project`.

- Run the following command-lines to download a code template for this lab

```
wget "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-CD0251EN-SkillsNetwork
/labs/m5_django_advanced/lab2_template.zip"
unzip lab2_template.zip
rm lab2_template.zip
```

Your Django project should look like the following:




- `cd` to the project folder:

```
cd lab2_template
```

- Install the necessary Python packages.

```
python3 -m pip install -U -r requirements.txt
```

Next activate the models for an `onlinecourse` app.

- Perform migrations to create necessary tables:

```
python3 manage.py makemigrations
```

- and run migration to activate models for `onlinecourse` app.

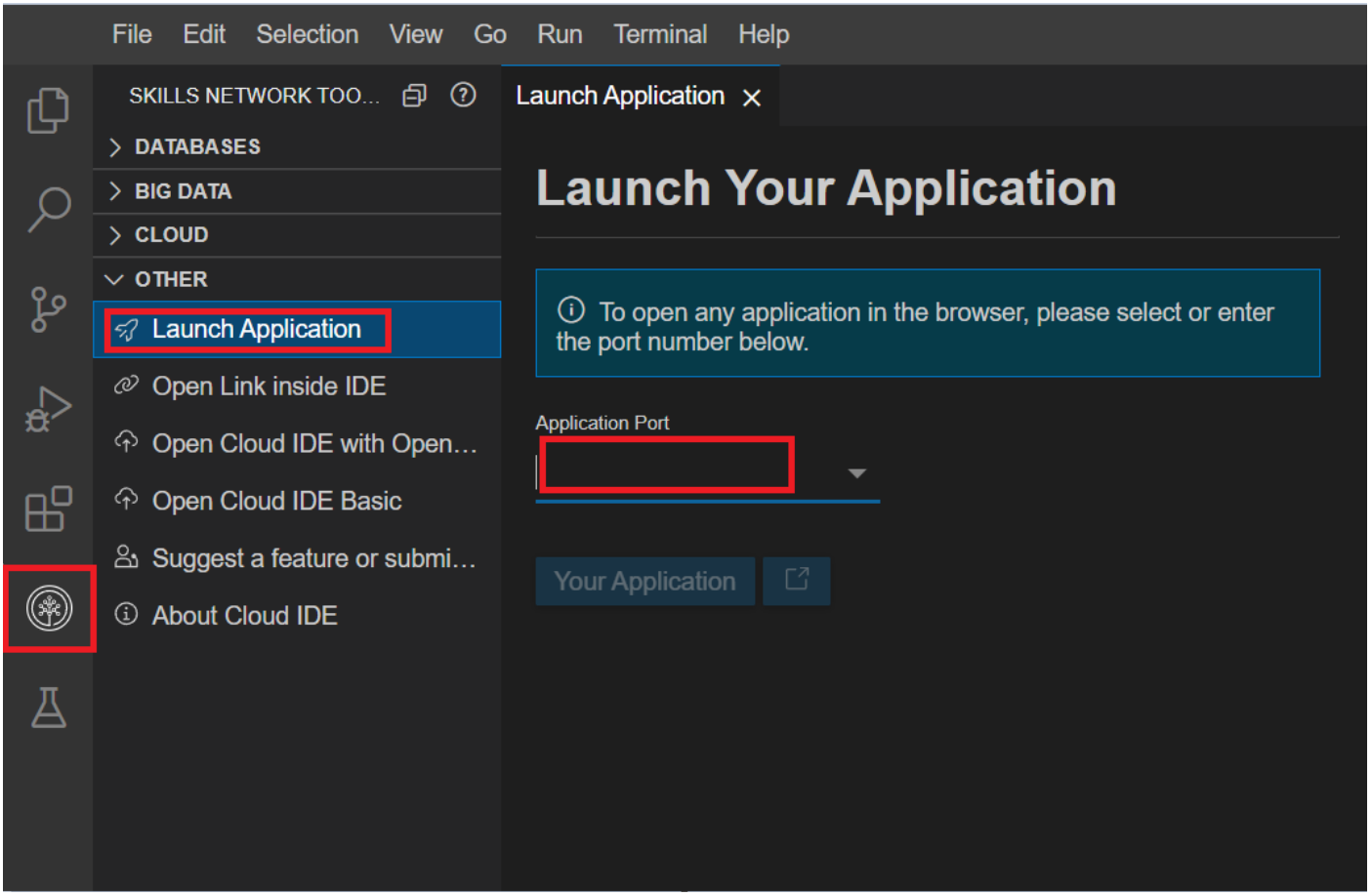
```
python3 manage.py migrate
```

Now let's test the imported `onlinecourse` app.

- Start the development server

```
python3 manage.py runserver
```

- Click on the Skills Network button on the left, it will open the **"Skills Network Toolbox"**. Then click the **Other** then **Launch Application**. From there you should be able to enter the port `8000` and launch.




When the browser tab opens, add the `/onlinecourse` path and your full URL should look like the following:

`https://userid-8000.theiadocker-1.proxy.cognitiveclass.ai/onlinecourse`

Now you should see the `onlinecourse` app started with a list of courses as the main page.

# Display User Information

Django authentication system is a Django built-in app used for managing user authentication and authorization.

Let's start learning Django authentication by quickly creating a superuser and displaying its user information on the main page.

- Stop the server if started and run:

```
python3 manage.py createsuperuser
```

With `Username`, `Email`, `Password` entered, you should see a message indicates the super user is created:

```
Superuser created successfully.
```

Let's start the server again and login with the super user.

```
python3 manage.py runserver
```

- Click `Launch Application` and enter the port for the development server `8000`

After the browser tab opens, add the `/admin` path and your full URL should look like the following

`https://userid-8000.theiadocker-1.proxy.cognitiveclass.ai/admin`

- Log in admin site with the credential you just created for the super user.
- Click the `Users` under `AUTHENTICATION AND AUTHORIZATION` section, and find the superuser

you just created

- Try to add some profile information such as `First name`, `Last name` and so on which will be shown on the main page.

Next, let's try to retrieve the superuser and show its profile on `onlinecourse/course_list.html` template

- Open `onlinecourse/templates/onlinecourse/course_list.html`, add the following code snippet under comment `<!-- Authentication section-->`

```
{% if user.is_authenticated %}
<p>Username: {{user.username}}, First name: {{user.first_name}}, Last name: {{user.last_name}} </p>
{% endif %}
```

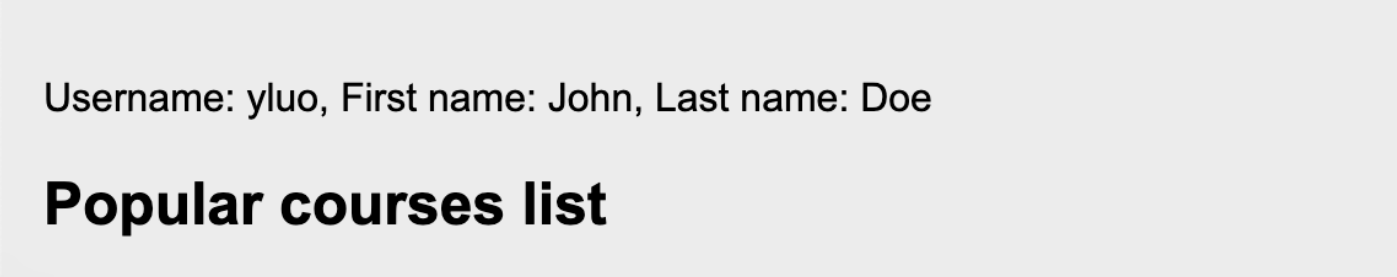
In the template, the `user` object will be queried by Django automatically for you based on the `session_id` created after login, and it will be available to both templates and views.

Then, we will check if the user is authenticated or if it is an anonymous user by using a if tag `{% if user.is_authenticated %}` and display the profile such as first name, last name, email, etc if authenticated.

To test, make sure the development server is running and go to:

`https://userid-8000.theiadocker-1.proxy.cognitiveclass.ai/onlinecourse`.

You should see the user information on the top.




# Login and Logout

Once you log in the superuser, your browser will keep `session_id` in cookie so that the superuser remaining logged in until the session expired.

Next, let's try to logout the superuser manually from main page by adding a logout dropdown button.

- Open `onlinecourse/course_list.html`, update the code between `{% if user.is_authenticated %}` and `{% endif %}` with a

dropdown `<div>`:

```
{% if user.is_authenticated %}
<div class="rightalign">
       <div class="dropdown">
           <button class="dropbtn">{{user.first_name}}</button>
           <div class="dropdown-content">
                <a href="{% url 'onlinecourse:logout' %}">Logout</a>
           </div>
       </div>
</div>
{% endif %}
```

The above code block adds a dropdown button to display the user's first name. When you hover the button, it pops up a link referring to a

logout view.

Next, let's create the `logout_request` view.

- Open `onlinecourse/views.py`, add a function-based logout view under the comment `# Create authentication related views`:

```
def logout_request(request):
    # Get the user object based on session id in request
    print("Log out the user `{}`".format(request.user.username))
    # Logout user in the request
    logout(request)
    # Redirect user back to course list view
    return redirect('onlinecourse:popular_course_list')
```

The above code snippet calls a built-in `logout` method with the `request` as an argument to log out the user (obtained from the request).

- Configure a route for `logout_request` view by adding a path entry in `urlpatterns`:

```
path('logout/', views.logout_request, name='logout'),
```

You can now test the logout functionality by refreshing the course list page. After you click `logout` button from the drop down, you should see the dropdown button disappeared because the user was not authenticated anymore.

Next, let's try to log in the superuser again.

- Open `templates/onlinecourse/course_list.html`, update the `{% if user.is_authenticated %}` block

```
{% if user.is_authenticated %}
    <div class="rightalign">
        <div class="dropdown">
            <button class="dropbtn">{{user.first_name}}</button>
            <div class="dropdown-content">
                <a href="{% url 'onlinecourse:logout' %}">Logout</a>
            </div>
        </div>
    </div>
    {% else %}
    <div class="rightalign">
        <div class="dropdown">
            <button class="dropbtn">Visitor</button>
            <div class="dropdown-content">
                <a href="{% url 'onlinecourse:login' %}">Login</a>
            </div>
        </div>
    </div>
{% endif %}
```

Here, we added a `{% else %}` tag to handle the scenario when user is not authenticated and also created a new dropdown button with a link pointing to a `login` view.

The `login` view should return a common login page asking for user credential such as username and password.

Next, let's create a template for such `login` view:

- Open the `templates/onlinecourse/user_login.html`, and add a simple `form` to accept user name and password

```
<form action="{% url 'onlinecourse:login' %}"  method="post">
  {% csrf_token %}
  <div class="container">
    <h1>Login</h1>
    <label for="username"><b>User Name</b></label>
    <input type="text" placeholder="Enter User Name: " name="username" required>
    <label for="psw"><b>Password</b></label>
    <input type="password" placeholder="Enter Password: " name="psw" required>
    <div>
      <button class="button" type="submit">Login</button>
    </div>
  </div>
</form>
```

The key elements of this login form are two input fields for user name and password. After form submission, it sends a `POST` request to a login view.

Next, let's create a login view to handle login request.

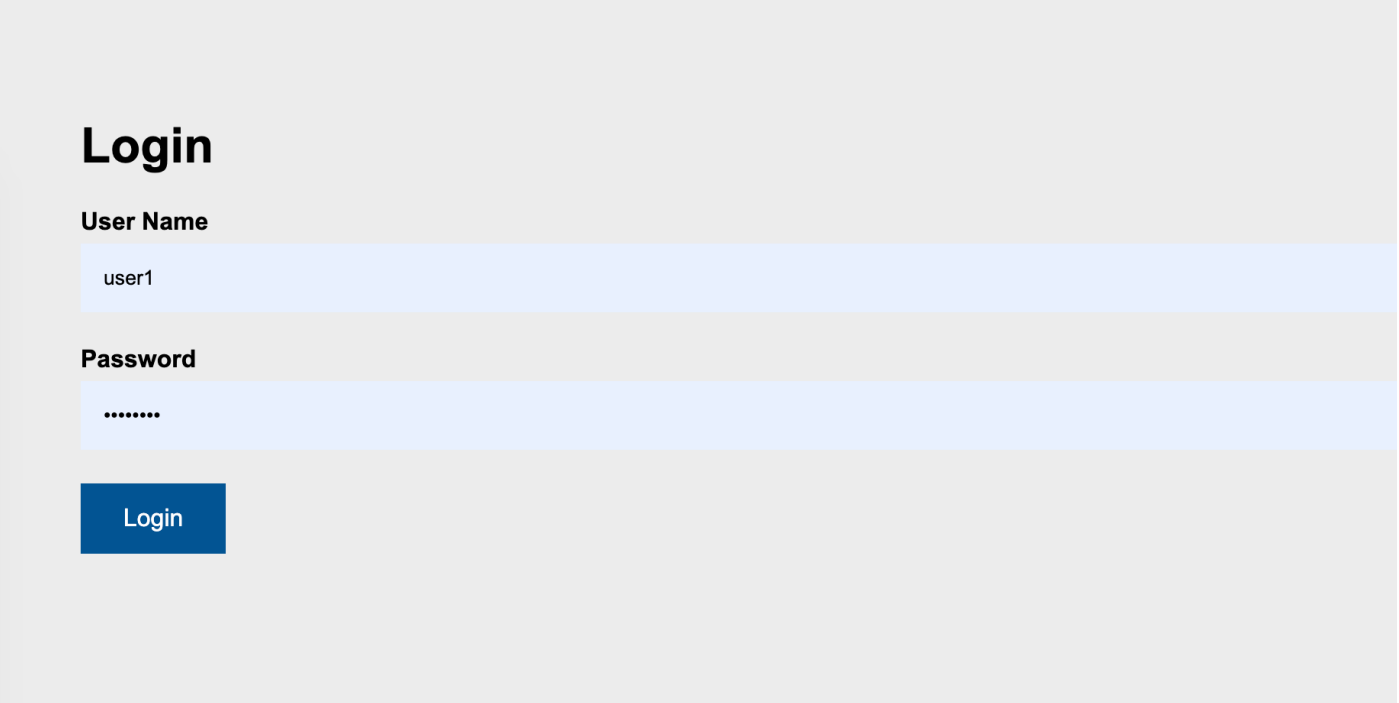
- Open `onlinecourse/views.py`, add a `login_request` view:

```
def login_request(request):
    context = {}
    # Handles POST request
    if request.method == "POST":
        # Get username and password from request.POST dictionary
        username = request.POST['username']
        password = request.POST['psw']
        # Try to check if provide credential can be authenticated
        user = authenticate(username=username, password=password)
        if user is not None:
            # If user is valid, call login method to login current user
            login(request, user)
            return redirect('onlinecourse:popular_course_list')
        else:
            # If not, return to login page again
            return render(request, 'onlinecourse/user_login.html', context)
    else:
        return render(request, 'onlinecourse/user_login.html', context)
```

- and configure a route for the `login_request` view by adding a path entry in `urlpatterns` list in `onlinecourse/urls.py`:

```
path('login/', views.login_request, name='login'),
```

Now we can test the login function by refreshing the course list page:




You could try both login and logout with the created superuser.

# Coding Practice: User Registration

In the previous step, we used the CLI to create a superuser. For regular users, we will need to create a user registration template and view to receive and save user credentials.

At the model level, a user object will be created in the `auth_user` table to complete the user registration. After the user is created, we can log in the user and redirect the user to the course list page.

- Open `onlinecourse/templates/onlinecourse/course_list.html`, update the authentication section by adding a

link `<a href="{% url 'onlinecourse:registration' %}">Signup</a>` pointing to a `registration` view to be created.

```
{% if user.is_authenticated %}
    <div class="rightalign">
        <div class="dropdown">
            <button class="dropbtn">{{user.first_name}}</button>
            <div class="dropdown-content">
                <a href="{% url 'onlinecourse:logout' %}">Logout</a>
            </div>
        </div>
    </div>
    {% else %}
    <div class="rightalign">
        <div class="dropdown">
            <form action="{% url 'onlinecourse:registration' %}" method="get">
                <input class="dropbtn"  type="submit" value="Visitor">
                <div class="dropdown-content">
                    <a href="{% url 'onlinecourse:registration' %}">Signup</a>
                    <a href="{% url 'onlinecourse:login' %}">Login</a>
                </div>
            </form>
        </div>
    </div>
{% endif %}
```

- Complete the following code snippet to create a `registration_request` view to handle a registration POST request:

```
def registration_request(request):
    context = {}
    # If it is a GET request, just render the registration page
    if request.method == 'GET':
        return render(request, 'onlinecourse/user_registration.html', context)
    # If it is a POST request
    elif request.method == 'POST':
        # <HINT> Get user information from request.POST
        # <HINT> username, first_name, last_name, password
        user_exist = False
        try:
            # Check if user already exists
            User.objects.get(username=username)
            user_exist = True
        except:
            # If not, simply log this is a new user
            logger.debug("{} is new user".format(username))
        # If it is a new user
        if not user_exist:
            # Create user in auth_user table
            user = User.objects.create_user(#<HINT> create the user with above info)
            # <HINT> Login the user and
            # redirect to course list page
            return redirect("onlinecourse:popular_course_list")
        else:
            return render(request, 'onlinecourse/user_registration.html', context)
```

▸ Click here to see solution

- Complete the following code snippet to create a `registration` template to accept user information:

```
<form action="{% url 'onlinecourse:registration' %}" method="post">
  <div class="container">
    <h1>Sign Up</h1>
    <hr>
    <!-- HINT, added inputs for username, firstname, lastname, and password  -->
    <div>
        {% csrf_token %}
        <button class="button" type="submit">Sign Up</button>
    </div>
  </div>
</form>
```

▸ Click here to see solution

- Add a route for the `registration_request` view by adding a path entry in `urlpatterns` list in `urls.py`:

```
path('registration/', views.registration_request, name='registration'),
```

Now refreshing the main page and try new registration function along with login/logout.

# Summary

In this lab, you have learned how Django authentication works and also created user login/logout and registration templates and views.

## Author(s)

[Yan Luo](linkedin.com/in/yan-luo-96288783)

## Changelog

| Date | Version | Changed by | Change Description |
|---|---|---|---|

| Date | Version | Changed by | Change Description |
| --- | --- | --- | --- |
| 14-Dec-2020 | 1.0 | Yan Luo | Initial version created |